VM guide: how the whole thing works and how it is deployed

A quick, end-to-end picture. It covers **today** (Postgres; the API runs charts itself), **after phase 4** (Service Bus queue and workers), and the release path from the Mac to the client VM. Detail lives in:

- FLOW.md: what each stage does;
- API.md: environment, models and the API;
- deploy/README.md: the copy into Azure DevOps.

---

1. The pieces

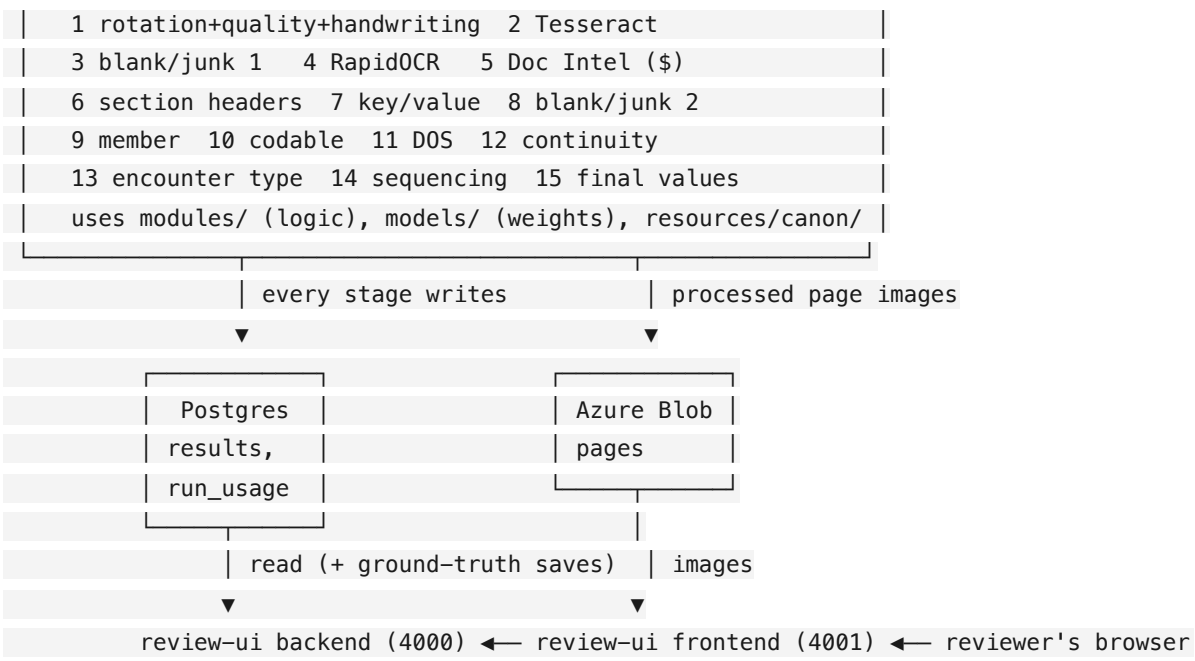
Piece	What it is	Runs where	Port
core-pipeline	The chart pipeline (15 stages) plus its API	Ubuntu VM, Docker	8001
review-ui backend	FastAPI over the database and Blob; ground-truth saves	Ubuntu VM, Docker	4000 (host) → 3000
review-ui frontend	React app served by nginx; talks only to the backend (`BACKEND_URL`)	Ubuntu VM, Docker (or any host)	4001 (host) → 3001
review-ui-client	The client's copy of review-ui, with its own DB	Separate deployment	5000 / 5001
db-utilities	One-off jobs: load manifests and ground truth into the database	Ubuntu VM, Docker (`docker compose run --rm`)	—
Postgres	Every result, ground truth, user, and `run_usage` monitoring	Azure / VM	5432
Azure Blob	Raw input pages, processed page images	Azure	—
Azure Doc Intelligence	Final OCR 2 (billed per page)	Azure	—
Service Bus `codingbusstd` / queue `chart-runs`	Job queue (created; used from phase 4)	Azure	—
Cosmos `medicalimagingtest`	Future database (reachable, empty; used from phase 2)	Azure	—

The deployments **share data, not code**. They never write into each other's folders. review-ui reads only the database and Blob.

---

2. Full flow of one chart (today)

```
 Playground (review-ui, admin) / curl / script
        |  POST /api/charts/run  or  /api/charts/batch-run
        ▼
 ┌──────────── core-pipeline container (port 8001) ────────────┐
 | apps/orchestrator_api  → 202 Accepted, chart_id             |
 |        | background task (BATCH_WORKERS charts at once,     |
 |        ▼                  STAGE_WORKERS pages at once)      |
 | apps/worker/runner.py  — the 15-stage chain                 |
 |   download pages from Blob → core-pipeline/outputs/charts/  |
```



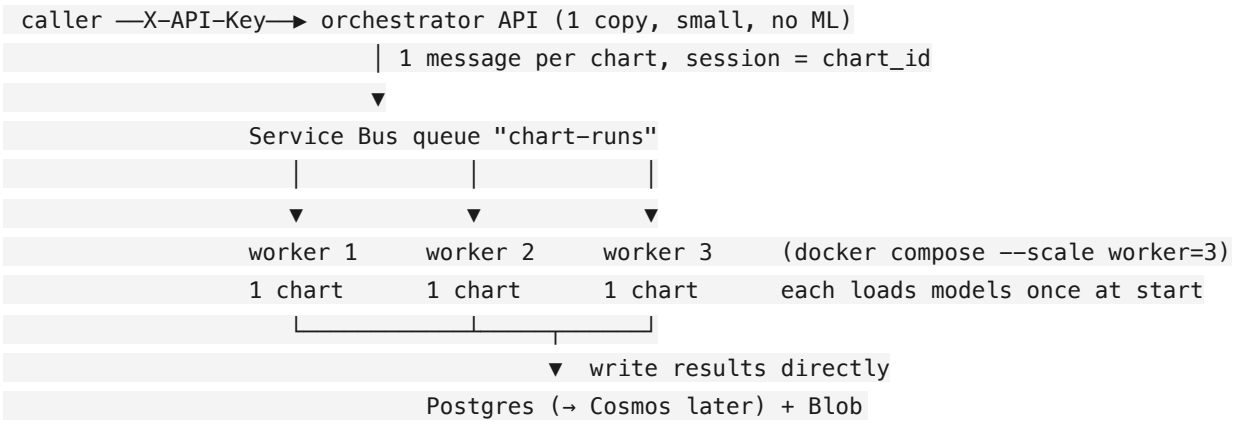
Step by step:

1. **Trigger.** An admin presses Run in the review-ui Playground, or someone calls the API with `curl`. The API answers `202` at once with the chart id.
2. **Intake.** The pages are copied from Blob (`input_path`) into `core-pipeline/outputs/charts/<chart>/`. That folder is scratch.
3. **The chain.** Stages 1–15 run in order. Each page's result is written to Postgres as the stage finishes. One bad page is recorded as an error, and the rest of the chart carries on.
4. **Monitoring.** When the chart ends, one `run_usage` row is written:
   - time and seconds per stage;
   - Doc Intel pages and cost;
   - LLM tokens (none yet).
5. **Review.** review-ui shows the chart from Postgres, with page images from Blob. Reviewers save ground truth back to Postgres.
6. **Housekeeping.** It runs once a day, on the first run of the day:
   - logs older than 7 days are deleted;
   - chart scratch folders not used for 7 days are deleted.

Progress: `GET /api/charts/{chart_id}`, or the chart list in review-ui.

---

3. Full flow after phase 4 (queue + workers)



- **The orchestrator does not process charts.** It checks the API key, records the run, puts one message per chart on the queue, and answers `202` with a `run_id`.
- **Workers pull.** Each worker takes one chart at a time. A free worker takes the next message, so a batch of 100 charts spreads itself over the workers. Add a worker and it simply starts pulling. No code changes and no restart of the others.
- **Sessions = chart id.** Two runs of the same chart are never processed at once.
- **A crash is safe.**
  - If a worker dies, its message returns to the queue after the 5-minute lock expires. Another worker picks it up and skips the stages that already finished.
  - After 5 failed deliveries the message goes to the dead-letter queue, where you can see it.
- **Workers write to the database themselves.** Results never pass back through the API.
- **Status:** `GET /v1/runs/{run_id}`, or `?wait=60` to wait for a small chart.

How many workers on the 16 CPU / 64 GB VM:

- one worker uses about 10–14 GB of memory (Docling, GLiNER and BERT) and about 4–5 CPUs;
- **start with 3**;
- go to 4 only if memory stays under 48 GB during a batch;
- a second VM later is just more workers on the same queue.

---

4. Where everything lives on the VM

```
~/advantmed-imaging-pipeline/            (Azure DevOps checkout)
├── core-pipeline/
│   ├── .env                  settings + secrets (never pushed)
│   ├── apps/                 orchestrator_api/, worker/ (runner, stages, jobs)
│   ├── common/  modules/     shared code / the logic
│   ├── resources/canon/      keyword JSONs (come with the code)
│   ├── models/               weights — copied by hand, never in git
│   ├── outputs/              scratch: charts/ (deleted after 7 unused days), debug/
│   └── logs/                 daily logs (deleted after 7 days)
├── review-ui/
│   ├── .env
│   ├── backend/  frontend/
├── db-utilities/
│   ├── .env                  DATABASE_URL + AZURE_STORAGE_* (as core-pipeline)
│   └── input/                files to load (mounted read-only at /input)
├── schema/                   schema.sql (new DB), migrate.sql (existing DB)
└── utilities/  docs/
```

`training/`, `scripts/`, `tests/` and `deploy/` are on GitHub only. They never reach the VM.

---

5. Release: Mac → GitHub → Windows → Azure DevOps → VM

`Mac ──git push──▶ GitHub ──Download ZIP──▶ Windows laptop ──push──▶ Azure DevOps ──git pull──▶ Ubuntu VM`

5.1 Windows laptop (PowerShell), each release

```
        # 1. github.com → Code → Download ZIP, then extract it
        Expand-Archive "$env:USERPROFILE\Downloads\model-repo-main.zip"
```

```
"$env:USERPROFILE\Downloads" -Force

        # 2. Copy into the DevOps folder (drops training/scripts/tests; keeps .env, models, logs)
        powershell -ExecutionPolicy Bypass -File "$env:USERPROFILE\Downloads\model-repo-main\deploy\prepare_for_devops.ps1" `
            -Source "$env:USERPROFILE\Downloads\model-repo-main" -Target "C:\work\advantmed-devops"

        # 3. Push C:\work\advantmed-devops to Azure DevOps as usual
```

5.2 Ubuntu VM (bash), each release

```
        cd ~/advantmed-imaging-pipeline
        git pull                                                    # from Azure DevOps

        # Database upgrade: safe to run every release
        export $(grep -E '^DATABASE_URL=' core-pipeline/.env | xargs)
        psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f schema/migrate.sql

        # core-pipeline
        cd core-pipeline
        docker compose up -d --build
        curl -fsS localhost:8001/ready && echo pipeline ready
        curl -s localhost:8001/health | python3 -m json.tool | head -40

        # review-ui (backend + frontend)
        cd ../review-ui
        docker compose up -d --build
        curl -fsS localhost:4000/docs >/dev/null && echo review-ui up
```

If `.env.example` gained a line, copy that line into the VM's `.env`, in the same section. The two files are kept line for line the same.

5.3 First-time setup on a new VM (once)

```
        sudo apt install -y docker.io docker-compose-v2 postgresql-client
        git clone <azure-devops-url> ~/advantmed-imaging-pipeline && cd ~/advantmed-imaging-pipeline

        cp core-pipeline/.env.example core-pipeline/.env   # fill in DB, Blob, Doc Intel keys
        cp review-ui/.env.example review-ui/.env           # same DATABASE_URL + Blob

        psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f schema/schema.sql   # empty database only

        # Model weights: from the Mac (or another host) into core-pipeline/models/
        #   hw/ rapidocr/ docling/ semantic-model/ ner/ blank-junk/ page-family/ encounter/
        scp -r models/* user@VM:~/advantmed-imaging-pipeline/core-pipeline/models/
```

From Windows the weights can be copied with `scp -r .\models\* user@VM:~/advantmed-imaging-pipeline/core-pipeline/models/`. OpenSSH comes with Windows 10 and 11. Weights are never pushed to git. From phase 8 they come from Blob automatically.

---

6. Running and watching

Task	Ubuntu VM (bash)	Windows laptop (PowerShell, VM at 172.20.4.170)
Health	`curl -s localhost:8001/health`	`curl.exe -s http://172.20.4.170:8001/health`
Run one chart	see below	same body with `curl.exe`
Chart progress	`curl -s localhost:8001/api/charts/by-name/<chart>`	`curl.exe -s http://172.20.4.170:8001/api/charts/by-name/<chart>`
Live logs	`docker compose logs -f api` (in `core-pipeline/`)	—
Log files	`core-pipeline/logs/`	—
Cost and time per day	`docker compose exec api python cli.py usage --days 7`	—
Stage list	`curl -s localhost:8001/api/stages`	`curl.exe -s http://172.20.4.170:8001/api/stages`

Run one chart from Blob:

```
curl -X POST localhost:8001/api/charts/run -H 'Content-Type: application/json' -d '{
  "input_type":"blob","container_name":"imaging-pipeline",
  "input_path":"Raw_Input/Run1/Batch1/DEID_PNGs",
  "chart_name":"52743839_44976074","output_path":"Processed/Run1"}'
```

Re-run only some stages, without paying for OCR again:

```
curl -X POST localhost:8001/api/charts/run -H 'Content-Type: application/json' -d '{
  "input_type":"blob","container_name":"imaging-pipeline",
  "input_path":"Raw_Input/Run1/Batch1/DEID_PNGs","chart_name":"52743839_44976074",
  "only":
["dos_extract","continuity","encounter_type","page_sequencing","imaging_final"]}'
```

`skip_ocr: true` re-runs everything except OCR, so there is no Doc Intel bill. A batch uses `/api/charts/batch-run` with `chart_list`. Most of the time the review-ui Playground does all of this for you.

6.1 A folder of charts, start to end (upload → run → review)

Example: the 9 test charts in `/Volumes/Extreme SSD/Advantmed/Training/Test Charts` (one subfolder per chart, the folder name is the chart name).

```
Mac / laptop folder ──upload──► Blob raw_input/Batch1/Run1/<chart>/*.jpg
                                        │  POST /api/charts/batch-run (VM, port 8001)
                                        ▼
                     VM core-pipeline: 15 stages ──► Postgres (all results)
                                        │              └► Blob Processed/Batch1/Run1/<chart>/
                                        ▼
                     review-ui (port 4001) shows the charts from Postgres + Blob
```

1. Upload the pages to Blob from the machine that has the folder. The script uses the Blob settings in `core-pipeline/.env` (`AZURE_STORAGE_ACCOUNT_NAME` must be set), skips files already uploaded, and `--dry-run` only lists what it would send.

```
        # Mac (or the VM, if the folder is there)
        cd core-pipeline && source .venv/bin/activate
        python ../utilities/upload_charts_to_blob.py "/Volumes/Extreme SSD/Advantmed/Training/Test Charts" \
          --prefix raw_input/Batch1/Run1

        # Windows laptop (needs Python 3.12 and core-pipeline\.venv; otherwise upload the
        # chart folders with Azure Storage Explorer into raw_input/Batch1/Run1/)
        cd core-pipeline; .venv\Scripts\Activate.ps1
        python ..\utilities\upload_charts_to_blob.py "D:\Test Charts" --prefix raw_input/Batch1/Run1
```

2. Run them on the VM. Every chart under the input path runs; results go to `Processed/Batch1/Run1/<chart>/`.

```
        # Ubuntu VM
        curl -X POST localhost:8001/api/charts/batch-run -H 'Content-Type: application/json' -d '{
          "input_type":"blob","container_name":"imaging-pipeline",
          "input_path":"raw_input/Batch1/Run1","output_path":"Processed/Batch1/Run1"}'

        # Windows laptop
        $body = '{"input_type":"blob","container_name":"imaging-pipeline","input_path":"raw_input/Batch1/Run1","output_path":"Processed/Batch1/Run1"}'
        Invoke-RestMethod -Method Post -Uri http://172.20.4.170:8001/api/charts/batch-run -ContentType 'application/json' -Body $body
```

3. Watch and review. `curl -s localhost:8001/api/charts/by-name/<chart>` shows the stage reached; `docker compose logs -f api` shows the run. When a chart finishes, it is in review-ui at `http://172.20.4.170:4001`. Blob paths are case-sensitive: `raw_input` and `Raw_Input` are different folders.

Debug mode (`KV_DEBUG`, `DOS_DEBUG` … in `.env`): the explanation files go to `core-pipeline/outputs/debug/`. From phase 3 they also go to Blob `debug/`. Turn it off on the VM unless you are chasing a problem.

---

7. When something is wrong

Symptom	Look at
`/ready` returns 503	Database unreachable, or `pipeline_stage` empty: run `schema/migrate.sql`
A feature silently does nothing	`/health`: it names what is missing (e.g. `member_ner.ready=false` = GLiNER weights not in `models/ner`)
Chart stuck or failed	`docker compose logs api`, `core-pipeline/logs/core-pipeline.log` (older days: `.log.<date>`), `GET /api/charts/by-name/<chart>` (`error_message` per page)
review-ui shows no image	Blob settings in `review-ui/.env` (`AZURE_STORAGE_*`); the image is read from Blob only
review-ui shows no chart	Both `.env` files must point at the **same** `DATABASE_URL`
Doc Intel cost jumped	`python cli.py usage --days 7`; use `skip_ocr` or `only` for re-runs
Out of memory	Lower `BATCH_WORKERS` / `STAGE_WORKERS` (today) or the worker count (phase 4)

---

8. What changes in later phases (and what you will do on the VM)

Phase	What changes for you
2: DB toggle	`DB_BACKEND=postgres|cosmos|both` in **both** core-pipeline and review-ui `.env` (already there and checked at start; `/health` shows `db_backend`). `both` writes to both stores and reads Postgres. Only `postgres` is wired today, so keep it until cutover
3: outputs and debug	Nothing to do. Scratch is wiped after each run, and debug also goes to Blob
4: queue and workers	`docker compose up -d --build --scale worker=3`; callers send `X-API-Key: <ORCHESTRATOR_API_KEY>` (use a long random value, 32+ characters); Playground sends it for you
7: Postgres → Cosmos	Run the migration utility once. `DB_BACKEND=both` for a week (both stores written), then `DB_BACKEND=cosmos` in both `.env` files
8: resources from Blob	No more `scp` of weights. A new model or canon version is published to Blob, and workers pick it up